INFORME TÉCNICO

Análisis y mejora de seguridad de una aplicación Android

Proyecto: SeguridadAndroid

Estudiante:

Diego Godoy

Johan Cofre

Adrian Anzoategui

Jeremy Rodriguez

Asignatura: Programación Android

Docente: Oscar Monardez

Institución: Santo Tomás

Fecha de entrega: 28/10/2026

1. Introducción

SeguridadAndroid permite registrar cuentas, iniciar sesión y administrar notas privadas. El análisis muestra controles de protección de credenciales, cifrado de notas y separación de datos por usuario. El APK release vigente obtuvo 73/100 en MobSF, con cero hallazgos altos y dos advertencias medias que requieren revisión contextual.

En este informe examinamos los riesgos de inyección SQL, comunicación insegura y exposición de datos sensibles. Diferenciamos la revisión del código, los hallazgos de la herramienta y las pruebas funcionales documentadas: la presencia de un control no equivale por sí sola a demostrar resistencia frente a todos los ataques.

2. Objetivos

2.1 Objetivo general

Analizar y documentar las vulnerabilidades de seguridad de una aplicación Android y las medidas aplicadas para mitigarlas.

2.2 Objetivos específicos

- Identificar y describir las vulnerabilidades contempladas en el README.md.
- Explicar las medidas técnicas aplicadas para reducir los riesgos.
- Documentar las pruebas realizadas y sus resultados con evidencias verificables.
- Revisar la documentación complementaria y proponer acciones de mejora continua.

3. Descripción del proyecto

Nombre del proyecto: Seguridad Android

Propósito y funcionalidad principal: La aplicación está dirigida a usuarios que necesitan guardar y consultar notas privadas en un dispositivo Android. Las cuentas y las notas se almacenan en SQLite. Google Maps y la consulta de ubicación complementan la aplicación; no existe un servidor propio de autenticación ni sincronización de notas.

Tecnologías utilizadas: Java, Android Studio, SQLiteOpenHelper, Android Keystore, AES-GCM, PBKDF2-HMAC-SHA256 y MobSF 4.6.2. La configuración del proyecto utiliza AGP 9.3.0, SDK de compilación y destino 37 y SDK mínimo 30.

Alcance del análisis: registro e inicio de sesión, sesión en memoria, creación y consulta de notas, eliminación filtrada por propietario, configuración del manifiesto y análisis estático del APK. Las capturas de interfaz de los anexos documentan login y notas; no documentan visualmente la pantalla del mapa.

4. Vulnerabilidades identificadas

4.1 Inyección SQL

Componente revisado: DatabaseHelper.java. MobSF informa una advertencia por el uso de SQLite y consultas SQL directas. Una consulta sería vulnerable si incorporara entradas externas como parte de su texto SQL, permitiendo alterar su lógica.

En los métodos revisados se separan los parámetros del texto SQL. La autenticación consulta el nombre de usuario mediante un marcador de posición:

"SELECT id, password_hash FROM usuarios WHERE username = ?",

new String[]{username}

Las notas se consultan con WHERE usuario_id = ?. La eliminación utiliza id = ? AND usuario_id = ?, y las inserciones emplean ContentValues. Las instrucciones execSQL revisadas crean tablas e índices con texto fijo.

Impacto potencial: lectura o modificación no autorizada de cuentas y notas si una futura consulta concatenara entradas. Medida observada: parametrización, validación y filtrado por propietario. Resultado P-01: control confirmado por revisión del código; no se documentó una prueba de explotación. La figura 8 identifica el componente, pero no muestra por sí sola todos sus métodos.

4.2 Comunicación no segura

Componente revisado: AndroidManifest.xml y configuración de Google Maps. El proyecto incluye acceso a Internet para los servicios de Google; las credenciales y las notas no se sincronizan con un servidor propio.

android:usesCleartextTraffic="false"

La configuración vigente expresa el rechazo de tráfico en claro para los componentes que respetan esa política. No se identificó en el código compartido un verificador personalizado que acepte cualquier certificado. No se implementó pinning de certificados.

Impacto potencial: exposición o modificación de datos en tránsito ante conexiones sin protección. Evidencia P-02: figura 10 y revisión del manifiesto. Resultado: política configurada; no se verificó mediante captura de tráfico, certificado no confiable o simulación de intermediario. No se afirma haber corregido una conexión HTTP previa, porque no existe evidencia de ella.

4.3 Exposición de datos sensibles

Datos revisados: contraseñas, contenido de notas, identificador de sesión y configuración de la API key. En SQLite se guardan password_hash y contenido_cifrado; la sesión se mantiene en memoria. Las coordenadas no se incorporan a la base de datos de notas.

PasswordUtils utiliza PBKDF2-HMAC-SHA256 con 600.000 iteraciones, sal aleatoria de 16 bytes y hash de 256 bits. La verificación utiliza MessageDigest.isEqual. Las contraseñas se derivan a un hash; no se cifran para recuperarlas después.

CryptoUtils utiliza AES-256-GCM y una clave generada en Android Keystore. El proveedor genera un IV aleatorio. El identificador del propietario se añade como dato autenticado adicional. Existe una clave de aplicación, no una clave independiente por cuenta; la autorización también depende de la sesión y del filtrado de consultas.

Se deshabilitó allowBackup y se excluyeron las bases de datos de las reglas de respaldo y transferencia. La API key se configura en local.properties, excluido de Git; se incorpora al APK al compilar. Sus restricciones de paquete, firma y API deben revisarse con su administrador y no se consideran verificadas.

Resultado P-03: las figuras 7 a 10 y la revisión del código respaldan la implementación. No se adjunta extracción de la base de datos ni auditoría completa de logs. La figura 5 demuestra guardado y lectura en la interfaz, no almacenamiento cifrado por sí sola.

5 Mejoras de seguridad implementadas

Área	Medida y componente	Evidencia y resultado
SQL	Parámetros y ContentValues en DatabaseHelper.	P-01: revisión del código; advertencia genérica permanece.
Credenciales	PBKDF2, sal y comparación de hashes en PasswordUtils.	P-03 y P-06: revisión y rechazo de credenciales incorrectas.
Notas	AES-GCM y Android Keystore en CryptoUtils.	P-03: implementación revisada; contenido visible solo tras descifrarlo en la app.
Entradas	Validación de usuario, contraseña y nota vacía.	P-04: nota vacía rechazada; figura 6.
Entrega	APK release firmado y respaldos restringidos.	MobSF: desaparecen avisos de debug y respaldo del análisis inicial.
Red	Rechazo de tráfico en claro en el manifiesto.	P-02: configuración observada; TLS no probado de forma independiente.

6. Plan de pruebas y evidencias

ficticias. Las capturas conservadas de login y notas pueden pertenecer a una compilación anterior a Maps; no se utilizan para atribuir al APK vigente pruebas no registradas.

ID	Prueba y procedimiento	Resultado esperado	Resultado obtenido y evidencia
P-01	Revisar argumentos de rawQuery y delete.	La entrada se trata como dato y se exige el propietario.	Revisión: consultas parametrizadas. Ataque con carga SQL pendiente. Código de DatabaseHelper y sección 4 1.
P-02	Revisar política de transporte y validar certificados.	Rechazar HTTP y certificados no confiables.	Parcial: usesCleartextTraffic=false, figura 10. Prueba de tráfico y TLS pendiente.

ID	Prueba y procedimiento	Resultado esperado	Resultado obtenido y evidencia
P-03	Revisar hashes, cifrado y almacenamiento.	No guardar contraseñas ni notas en texto legible.	Parcial: algoritmos y campos revisados, figuras 7 a 9. Extracción de SQLite y revisión de logs pendientes.
P-04	Dejar la nota vacía y pulsar Guardar nota.	Rechazar la entrada con un mensaje.	Satisfactorio: «Escribe una nota». Figura 6.
P-05	Abrir la app, crear cuenta, iniciar sesión y guardar una nota.	Completar el flujo y mostrar la nota.	Satisfactorio en capturas funcionales: figuras 1, 2 y 5. La pantalla de notas acredita acceso a la sesión.
P-06	Repetir registro con usuario existente y probar clave incorrecta.	Rechazar la operación y no conceder acceso.	Rechazo visible en figuras 3 y 4. El mensaje genérico de registro no prueba por sí solo la causa SQL del rechazo.
P-07	Cargar el APK release con Maps en MobSF.	Generar hallazgos y conservar un reporte identificable.	Ejecutado: 73/100; 0 altos, 2 medios. vulnerability_report.pdf y hashes de la sección 9.

7. Documentación complementaria

7.1 vulnerabilities.md

Debe registrar las vulnerabilidades detectadas, su descripción, impacto, evidencia y estrategia de mitigación.

7.2 best_practices.md

Debe reunir buenas prácticas de desarrollo seguro, manejo de datos, consultas SQL, autenticación, permisos y comunicaciones.

7.3 security_tips.md

Debe presentar recomendaciones concretas para el desarrollo, la configuración y el uso seguro de la aplicación.

7.4 security_improvement_program.md

Debe proponer un programa de mejora continua con acciones pendientes, prioridad, responsable, plazo y criterio de verificación.

El README solo menciona estos archivos; para afirmar su contenido exacto es necesario revisar los documentos reales del repositorio.

8. Procedimiento de ejecución segura

1. Clonar el repositorio desde la fuente oficial del proyecto.
2. Abrir o importar el proyecto en Android Studio y esperar la sincronización de Gradle.
3. Revisar las instrucciones de compilación, versiones de SDK y dependencias.
4. Ejecutar la aplicación en un emulador o dispositivo de prueba autorizado.
5. Verificar los permisos necesarios y conceder solo los que correspondan a la funcionalidad.

6. Comprobar las funciones principales y realizar las pruebas de seguridad documentadas.

Registrar versiones del entorno y cualquier error encontrado. No compartir archivos de configuración que contengan credenciales, tokens o claves.

9. Resultados

Indicador MobSF 4.6.2	APK debug inicial	Release sin Maps	Release vigente
Puntaje	45/100	73/100	73/100
Riesgo	Medio	Bajo	Bajo
Hallazgos altos	2	0	0
Hallazgos medios	3	2	2

Persisten las advertencias sobre ProfileInstallReceiver protegido por DUMP y el uso de SQLite. El resumen registra también un elemento secure y un hotspot; el PDF no detalla una regla concreta para este último. Los permisos de ubicación clasificados como dangerous requieren autorización del usuario y no prueban por sí mismos que la aplicación sea maliciosa.

10. Conclusiones

La aplicación incorpora controles concretos para contraseñas, notas y acceso a datos. El paso de debug a release y las restricciones de respaldo eliminaron advertencias presentes en el primer APK. La incorporación de Maps mantuvo el puntaje estático en 73/100, sin hallazgos altos.

Las capturas respaldan el registro, el rechazo de operaciones, la visualización de notas y la validación de una nota vacía. La revisión del código respalda la presencia de parametrización y mecanismos criptográficos.

11. Referencias

Repositorio y documentación del proyecto: https://github.com/diegogodoy21421/SeguridadAndroid.

MobSF 4.6.2. Reportes estáticos del APK inicial y de los APK release. Reporte vigente: vulnerability_report.pdf, 28-10-2026.

Android Developers. Seguridad y privacidad: https://developer.android.com/privacy-and-security

Android Developers. Android Keystore: https://developer.android.com/privacy-and-security/keystore

Google Maps Platform. Seguridad de API keys: https://developers.google.com/maps/api-security-best-practices

OWASP Mobile Application Security: https://mas.owasp.org/

Anexo A. Registro de evidencias

Registro e inicio de la aplicación

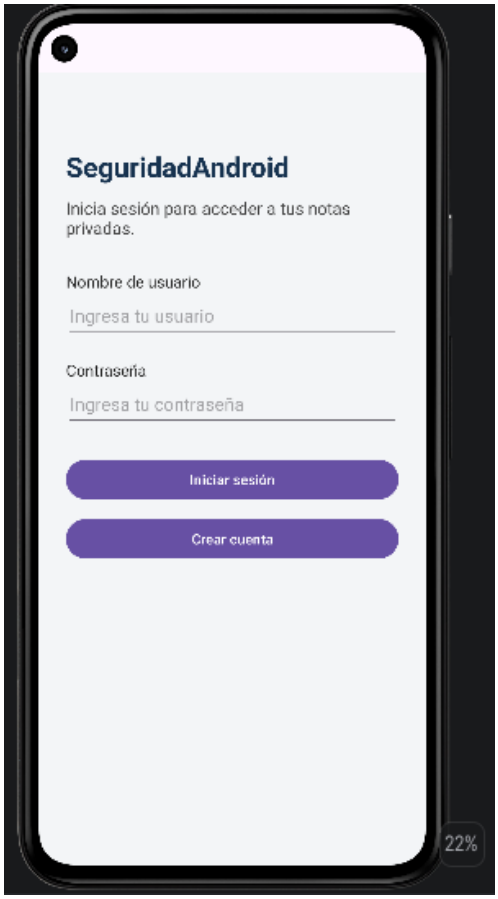


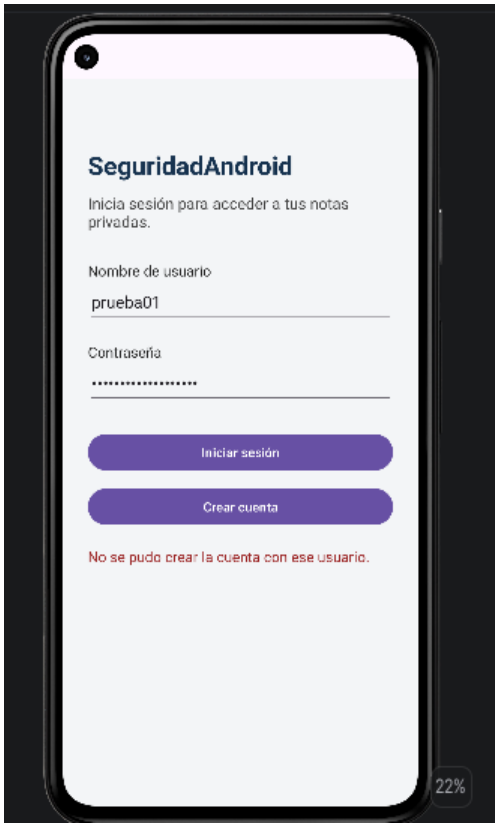


Figura 1. Pantalla de inicio con acceso a registro e inicio de sesión. No demuestra autenticación exitosa por sí sola.

Figura 2. Mensaje «Cuenta creada. Ya puedes iniciar sesión». Evidencia del flujo de registro con una cuenta de prueba.

Identificación: capturas funcionales conservadas durante el desarrollo. La cuenta prueba01 es un dato ficticio de prueba.

Anexo A Rechazo de operaciones



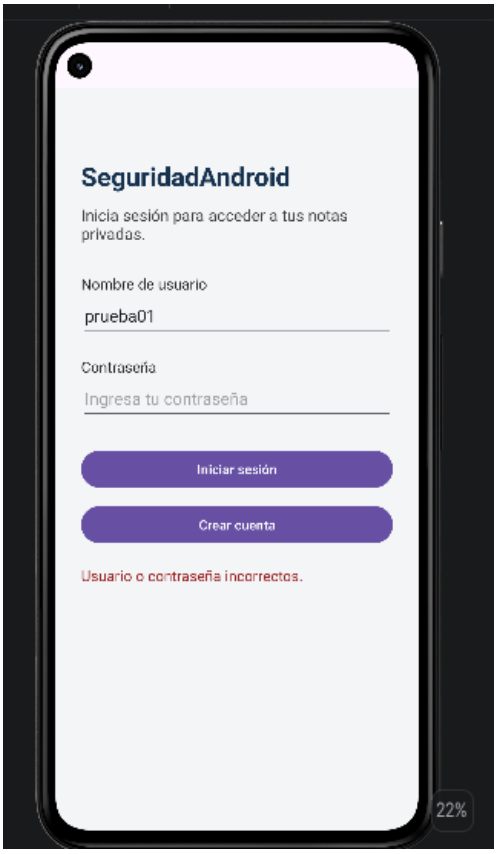


Figura 3. Intento de registro identificado como usuario duplicado. La app muestra un rechazo genérico; la restricción UNIQUE se verifica en el esquema del código.

Figura 4. Mensaje «Usuario o contraseña incorrectos». Evidencia de rechazo de credenciales y uso de un mensaje genérico.

Anexo A Guardado y validación de notas

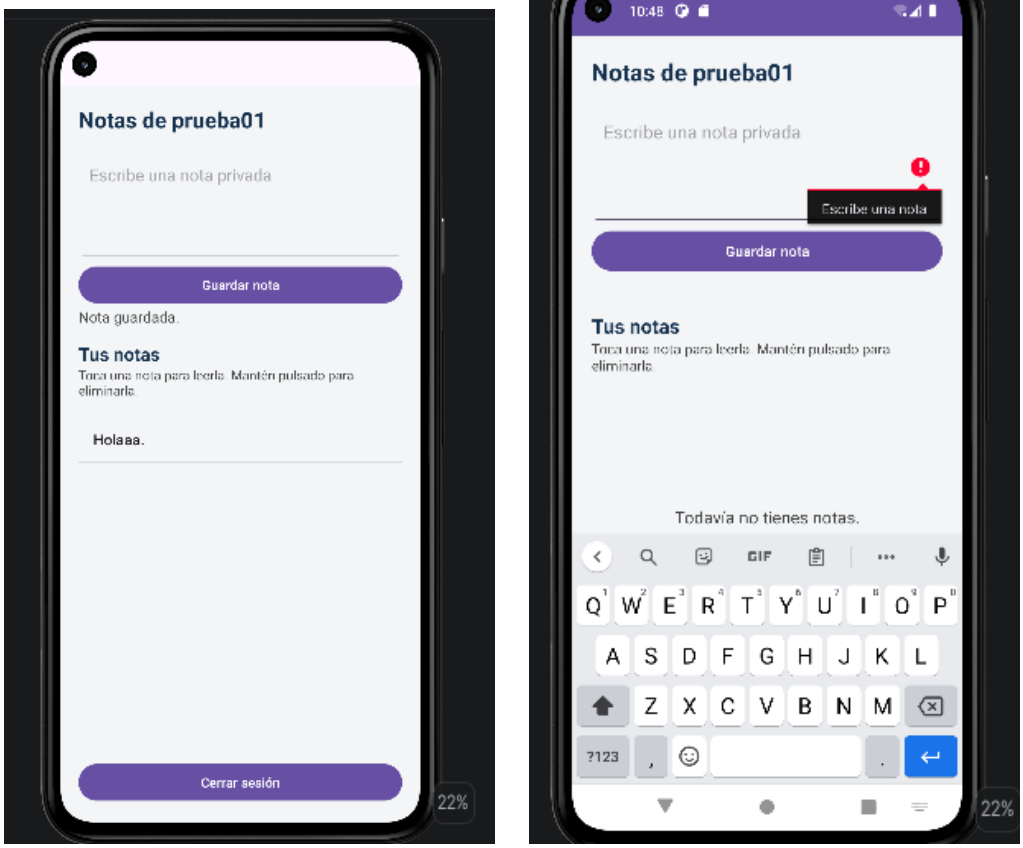


Figura 5. Nota visible y mensaje «Nota guardada». Evidencia de persistencia funcional en la interfaz, sin demostrar por sí sola el cifrado en SQLite.

Figura 6. Campo vacío rechazado con «Escribe una nota». Corresponde a P-04 y respalda la validación de entrada.

Anexo A Componentes de cifrado y persistencia

```
public final class CryptoUtils {

    3 usages
    private static final String ALIAS = "SeguridadAndroidNotas";
    2 usages
    private static final String PROVEEDOR = "AndroidKeyStore";
    2 usages
    private static final String TRANSFORMACION = "AES/GCM/NoPadding";

    no usages
    private CryptoUtils() {
    }

    2 usages
    private static synchronized SecretKey obtenerClave(boolean crear)
```

Figura 7. CryptoUtils declara AndroidKeyStore y AES/GCM/NoPadding. La captura identifica la configuración; el análisis de generación de IV y cifrado se basa en el código completo.

```
public class DatabaseHelper extends SQLiteOpenHelper {

    1 usage
    private static final String NOMBRE_BD = "seguridad_android.db";
    1 usage
    private static final int VERSION_BD = 2;

    2 usages
    public DatabaseHelper(Context context) {
        super(
                context.getApplicationContext(),
                NOMBRE_BD,
                factory: null,
                VERSION_BD
        );
    }

    5 usages
```

Figura 8. DatabaseHelper extiende SQLiteOpenHelper y define nombre y versión de base de datos. Las consultas parametrizadas se documentan en la sección 4 1.

Anexo A Contraseñas y configuración

```
public final class PasswordUtils {

    3 usages
    private static final int ITERACIONES = 600_000;
    2 usages
    private static final int LONGITUD_SAL = 16;
    2 usages
    private static final int LONGITUD_HASH_BITS = 256;

    no usages
    private PasswordUtils() {
    }

    1 usage
    public static String crearHash(char[] password)
            throws GeneralSecurityException {
```

Figura 9. PasswordUtils declara 600.000 iteraciones, sal de 16 bytes y hash de 256 bits. El algoritmo PBKDF2-HMAC-SHA256 se verifica en el método de derivación.

```
<application
    android:allowBackup="false"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:usesCleartextTraffic="false"
    android:theme="@style/Theme.SeguridadAndroid">

    <meta-data
        android:name="com.google.android.geo.API_KEY"
```

Figura 10. Manifiesto con allowBackup=false y usesCleartextTraffic=false. No se incluye el valor de la API key.

Anexo B. Datos de ejecución

Dato	Registro
Sistema anfitrión	Windows; versión exacta no registrada en las evidencias de esta entrega.
Entorno de desarrollo	Android Studio; número de versión no registrado.
Lenguaje	Java; compatibilidad de código fuente y destino Java 11. Esto no identifica la versión del JDK que ejecuta Gradle.

Dato	Registro
Construcción	Android Gradle Plugin 9.3.0; compileSdk 37; targetSdk 37; minSdk 30.
Emulador	MobSF_API30, Android 11, API 30; imagen Google APIs x86_64.
Herramienta	MobSF 4.6.2, ejecutado en Docker Desktop.
Fechas	Análisis inicial y desarrollo: 27-10-2026. Reporte release vigente y captura de validación: 28-10-2026. No todas las capturas conservan una fecha individual verificable.
Datos utilizados	Cuenta ficticia prueba01 y notas de ejemplo; sin datos personales reales.
Reporte vigente	vulnerability_report.pdf, generado a partir del APK release con Maps.
Clave de firma y API	No se incluyen contraseñas, claves privadas ni el valor de la API key en este informe.